

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases

Functions – Case Based Learning

Name: K. Dedeepya

Reg no: 192324252

1. A manufacturing industry needs to predict machine output based on sensor readings that vary over time. Propose a solution using Locally Weighted Regression. Explain how local models improve prediction accuracy and how the system handles noisy or incomplete data.

Problem: A manufacturing industry needs to predict machine output based on sensor readings that vary over time.

Proposed Solution: Locally Weighted Regression (LWR)

Locally Weighted Regression is a non-parametric regression technique that gives more importance to training data points that are close to the current input. This is useful in manufacturing because machine behaviour can change with temperature, vibration, pressure, speed, and operating time.

Input Features

Sensor Feature	Example
Temperature	75°C
Pressure	120 kPa
Vibration	3.2 mm/s
Machine Speed	1500 RPM
Operating Time	5 hours
Output	850 units

Working

1. Collect historical sensor readings and machine output.
2. Clean the sensor data and handle missing values.
3. Normalize the features because sensors may have different scales.
4. For a new sensor reading, calculate its distance from historical observations.
5. Assign higher weights to nearby observations.
6. Build a local regression model using these weighted observations.

7. Predict the machine output.
8. Continuously update the model using newly collected sensor data.

Weight Calculation

A commonly used Gaussian weight is:

$$w_i = e^{-\frac{(x-x_i)^2}{2\tau^2}}$$

Where:

- x = current input
- x_i = historical data point
- w_i = weight assigned to the data point
- τ = bandwidth parameter

Closer data → higher weight

Farther data → lower weight

How Local Models Improve Accuracy

A single global regression model assumes that the relationship between sensor readings and output is similar everywhere. However, machine behaviour may change under different operating conditions. LWR creates a model around the current operating condition. Therefore, it can capture local changes such as:

- High temperature causing lower production.
- Increased vibration indicating machine degradation.
- Different output behaviour at different machine speeds.

Handling Noisy Data

Noisy sensor readings can affect predictions. The system can:

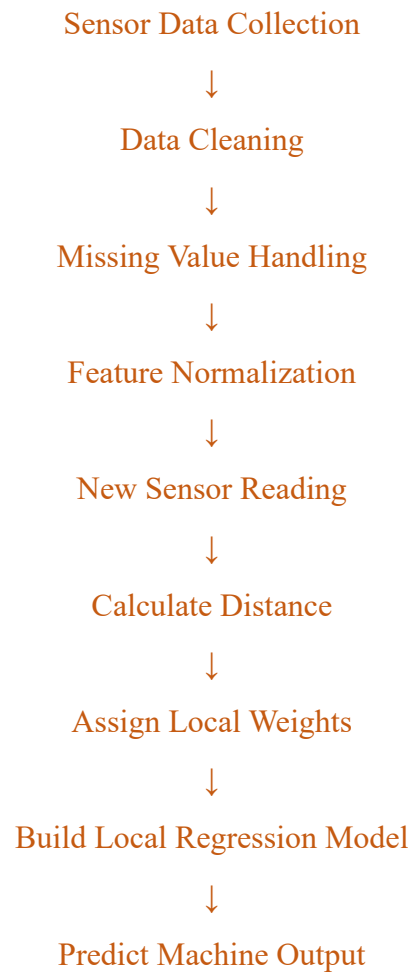
- Remove extreme outliers.
- Apply moving-average or median filtering.
- Normalize sensor values.
- Give lower influence to distant observations.

Handling Incomplete Data

Missing sensor values can be handled using:

- Mean/median imputation.
- Interpolation for time-series readings.
- Previous valid sensor values.
- Removing records with excessive missing values.

Flowchart



Conclusion

Locally Weighted Regression is suitable for manufacturing because it focuses on nearby operating conditions rather than assuming one fixed relationship for the entire dataset. This allows the system to provide more accurate predictions when machine behaviour changes over time.

2. A banking system wants to detect potential loan defaulters based on customer financial history and transaction patterns. Design a solution using K-Nearest Neighbour or Logistic Regression. Explain how similarity or decision boundaries are used, how features are selected, and how the system adapts to new customer data over time.

Problem: A banking system wants to identify potential loan defaulters using customer financial history and transaction patterns.

Proposed Solution: Logistic Regression

Logistic Regression is a supervised classification algorithm that predicts the probability of an event. Here, the target variable can be:

- **0 → non-defaulter**
- **1 → Defaulter**

It is suitable for banking because it is computationally efficient, scalable and provides interpretable probabilities.

Important Features

Feature	Example
Annual Income	₹6,00,000
Credit Score	720
Existing Loan Amount	₹2,00,000
Debt-to-Income Ratio	35%
Number of Previous Defaults	0
Monthly Transactions	45
Account Age	5 years

Working

1. Collect historical customer and loan data.
2. Label customers as defaulters or non-defaulters.
3. Clean the data and handle missing values.
4. Select relevant financial and transaction features.
5. Normalize numerical features where appropriate.
6. Train the Logistic Regression model.
7. Calculate the probability of default for a new customer.
8. Apply a classification threshold.
9. Classify the customer as high-risk or low-risk.

Logistic Regression Formula

The model calculates:

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

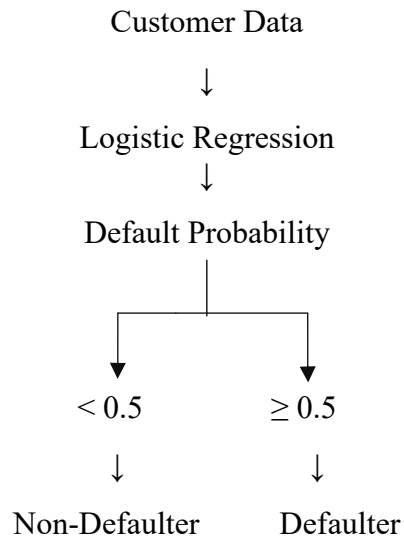
The output is a probability between 0 and 1.

For example:

Customer Default Probability = 0.82

Decision Boundary

Logistic Regression creates a decision boundary separating the two classes.



Feature Selection

Important features can be selected using:

- Correlation analysis.
- Statistical significance.
- Recursive Feature Elimination.
- Domain knowledge.

Adapting to New Customer Data

The banking system can periodically retrain the model using:

- New loan records.
- Recent transaction patterns.
- New repayment information.
- Updated credit scores.

This allows the model to adapt to changing financial behaviour.

Accuracy Improvement

Accuracy can be improved through:

Technique	Purpose
Feature selection	Remove irrelevant variables
Data balancing	Handle unequal default/non-default classes
Cross-validation	Test model reliability
Hyperparameter tuning	Improve model performance
Regularization	Reduce overfitting
Precision/Recall	Evaluate classification quality

For loan default detection, precision, recall, F1-score and ROC-AUC are generally more informative than accuracy alone when the classes are imbalanced.

Conclusion

Logistic Regression provides an efficient and interpretable solution for loan-default prediction. Its probability-based decision boundary allows banks to identify high-risk customers while continuously improving the model using new financial data.

3. A streaming platform aims to recommend movies to users based on viewing history and ratings. Propose a solution using KNN or Case-Based Learning. Explain how user similarity is calculated, how recommendations are generated, and how the system handles large-scale user data efficiently.

Problem: A streaming platform wants to recommend movies based on user viewing history and ratings.

Proposed Solution: K-Nearest Neighbour (KNN)

KNN can be used as a collaborative filtering recommendation technique. It identifies users who have similar movie preferences and recommends movies that similar users have watched or rated highly.

Example User-Movie Matrix

User	Movie A	Movie B	Movie C	Movie D
User 1	5	4	1	2
User 2	5	4	1	2
User 3	1	2	5	4
User 4	4	5	2	1

If User 1 is the target user, User 2 is highly similar.

Similarity Calculation

For recommendation systems, Cosine Similarity can be used:

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

A value close to 1 indicates high similarity.

Alternatively, Euclidean distance can be used:

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

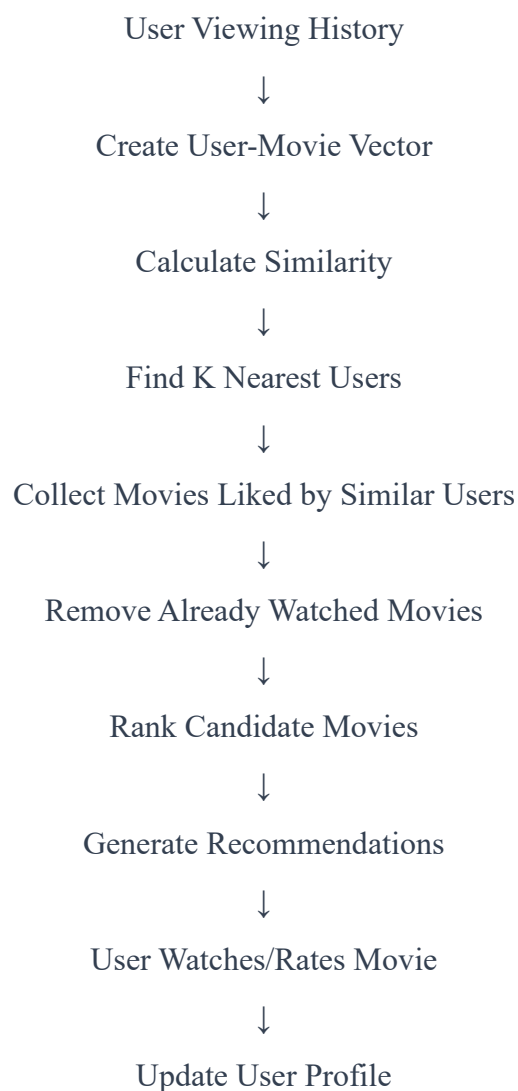
For KNN, smaller distance means greater similarity.

Working

1. Collect user viewing history and movie ratings.

2. Create user-movie feature vectors.
3. Normalize ratings if required.
4. Calculate similarity between users.
5. Select the K most similar users.
6. Identify movies highly rated by those users.
7. Remove movies already watched by the target user.
8. Rank the remaining movies.
9. Recommend the top-ranked movies.

Recommendation Flowchart



Example

Suppose: Target User $\rightarrow$ User 1

Nearest Users:

User 2 $\rightarrow$ Similarity = 0.95

User 4 → Similarity = 0.88

User 3 → Similarity = 0.30

Movies highly rated by User 2 and User 4 but not watched by User 1 can be recommended.

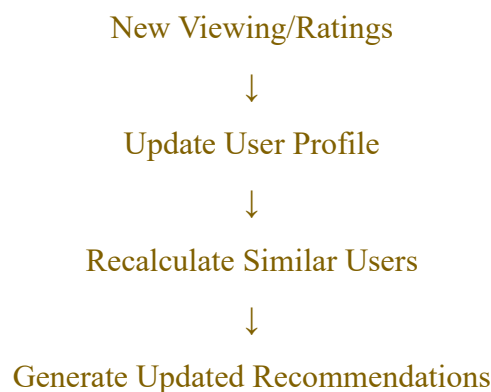
Handling Large-Scale Data.

The system can improve scalability using:

- User/item filtering.
- Sparse matrices.
- Approximate nearest-neighbour search.
- Dimensionality reduction.
- Batch processing.

Handling New User Data

When a user watches or rates a movie, the system updates the user's preference vector.



Advantages

Feature	Benefit
Simple algorithm	Easy to implement
Similarity-based	Captures user preferences
No complex training	Easy to update
Dynamic	Recommendations change with new ratings
Personalized	Different users receive different results

Conclusion

KNN-based collaborative filtering can recommend movies by finding users with similar viewing and rating patterns. By using efficient similarity search and sparse representations, the system can be scaled to large streaming platforms.

4. An energy management company aims to predict electricity consumption based on factors such as time, weather, and appliance usage. Propose a solution using Locally Weighted Regression or RBF networks. Explain how local learning captures variations in consumption patterns and how the model handles noisy and seasonal data.

Problem: An energy management company wants to predict electricity consumption based on time, weather and appliance usage.

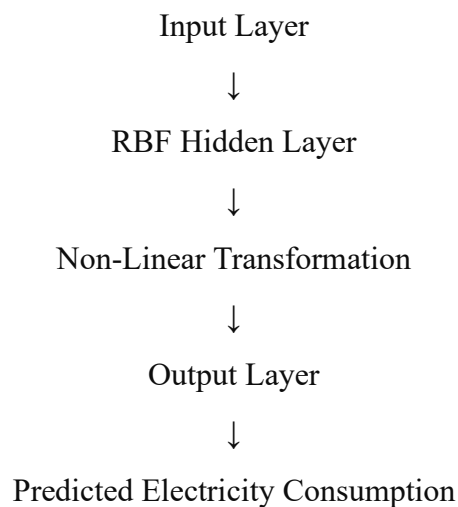
Proposed Solution: Radial Basis Function (RBF) Network

An RBF Neural Network is suitable because electricity consumption has non-linear relationships with factors such as temperature, time of day, weather and appliance usage. For example, electricity usage may increase significantly when temperature becomes very high because air-conditioner usage increases.

Input Features

Feature	Example
Hour	18:00
Day	Monday
Temperature	32°C
Humidity	70%
Appliance Usage	8 appliances
Previous Consumption	4.5 kWh
Output	Predicted electricity consumption

RBF Network Structure



RBF Function

A commonly used Gaussian radial basis function is:

$$\phi(x) = e^{-\frac{\|x-c\|^2}{2\sigma^2}}$$

Where:

- x = input vector
- c = centre of the RBF neuron
- σ = spread parameter

- $\phi(x)$ = activation value

The output can be represented as:

$$y = \sum_{j=1}^m w_j \phi_j(x) + b$$

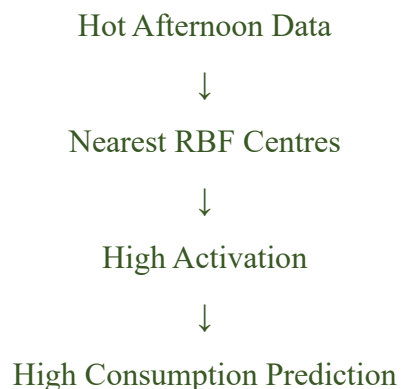
Working

1. Collect historical electricity consumption data.
2. Collect weather and appliance usage information.
3. Clean and preprocess the dataset.
4. Handle missing values and noisy readings.
5. Select important features.
6. Determine RBF centres using clustering such as K-Means.
7. Calculate RBF activations.
8. Train the output layer.
9. Predict future electricity consumption.
10. Update the model with new consumption data.

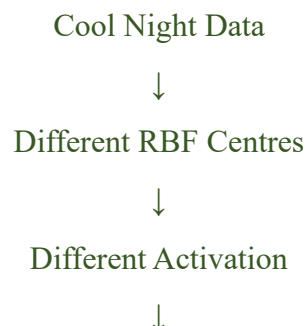
How Local Learning Captures Variations

RBF neurons respond strongly to inputs close to their centres.

For example:



While:



Handling Seasonal Data

The model can include:

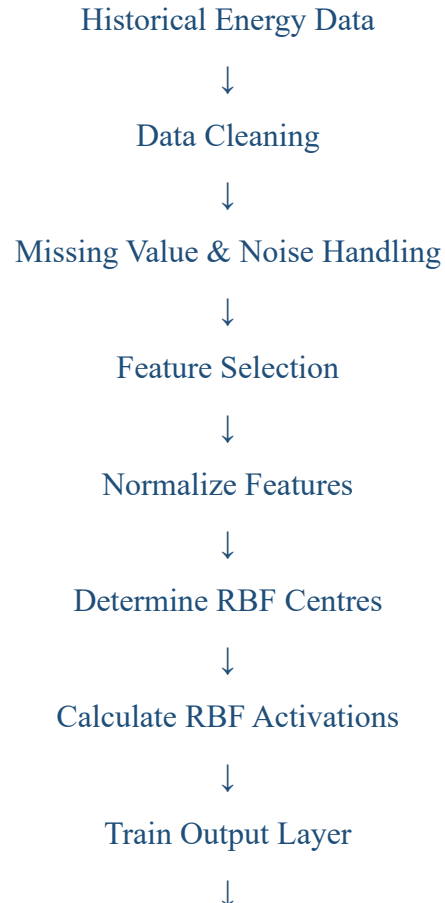
- Month.
- Season.
- Temperature.
- Day of week.
- Holiday indicator.
- Time of day.

Handling Noisy Data

Noise can be reduced using:

- Outlier detection.
- Moving averages.
- Data smoothing.
- Sensor validation.
- Missing-value imputation.

Flowchart



Predict Consumption



Evaluate Prediction



Update with New Data

Conclusion

RBF networks are suitable for electricity prediction because they can model complex and non-linear relationships. Local activation allows the system to learn different consumption patterns for different weather, time and appliance conditions.

5. A smart city system aims to predict traffic flow and congestion levels using data from sensors, weather, and time of day. Design a solution using Locally Weighted Regression or KNN. Explain how the model captures dynamic patterns, how predictions are made in real-time, and how scalability is maintained with large volumes of data.

Problem: A smart city system wants to predict traffic flow and congestion using sensor data, weather and time of day.

Proposed Solution: Locally Weighted Regression (LWR)

Locally Weighted Regression is appropriate because traffic conditions change dynamically. Traffic at 8 AM on a weekday can behave very differently from traffic at 11 PM on a weekend. Instead of creating one fixed global model, LWR focuses on historical observations similar to the current traffic situation.

Input Features

Feature	Example
Time	8:30 AM
Day	Monday
Vehicle Count	1250
Average Speed	28 km/h
Rainfall	5 mm
Road Occupancy	78%
Previous Traffic Flow	1100 vehicles/hour
Output	Future traffic flow

Working

1. Collect real-time traffic sensor data.
2. Collect weather and time information.
3. Clean and preprocess the data.
4. Normalize the input features.

5. Receive the current traffic condition.
6. Calculate similarity/distance with historical observations.
7. Assign higher weights to similar observations.
8. Build a local regression model.
9. Predict traffic flow or congestion.
10. Update predictions continuously as new sensor data arrives.

Weight Calculation

A Gaussian weight can be used:

$$w_i = e^{-\frac{d(x, x_i)^2}{2\tau^2}}$$

Where:

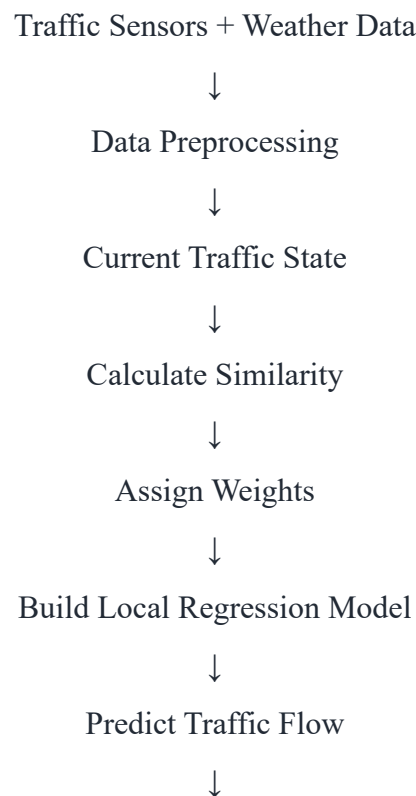
- $d(x, x_i)$ = distance between current and historical traffic conditions.
- w_i = weight of historical observation.
- τ = bandwidth.

Therefore:

Similar traffic condition → high weight

Different traffic condition → low weight

Flowchart



Determine Congestion Level



Real-Time Traffic Decision



New Sensor Data



Update Prediction

Example

Suppose the system receives:

Time = 8:30 AM

Day = Monday

Rainfall = 5 mm

Vehicles = 1250

Historical Condition	Similarity	Weight
Monday 8:20 AM, Rain	High	High
Monday 8:40 AM, Rain	High	High
Sunday 2:00 PM, No Rain	Low	Low
Saturday 11:00 PM	Very Low	Very Low

The nearby observations have greater influence on the prediction.

Dynamic Pattern Handling

Traffic changes because of:

- Peak hours.
- Weather.
- Accidents.
- Holidays.
- Weekends.
- Road construction.
- Special events

Real-Time Prediction

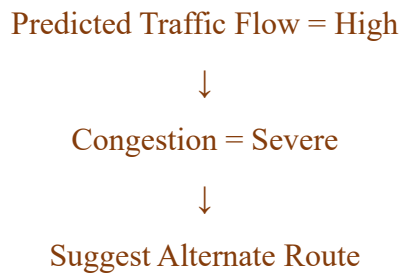
Real-Time Sensor



Current Traffic Features



For example:



Scalability

For large smart-city datasets, the system can use:

- Sampling.
- Sliding time windows.
- Spatial filtering.
- Efficient nearest-neighbour search.
- Distributed processing.

Performance Evaluation

The prediction system can be evaluated using:

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

and

$$RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$$

Where:

- y_i = actual traffic flow.

- $\hat{y}_i$ = predicted traffic flow.

Lower MAE and RMSE indicate better prediction performance.

Conclusion

Locally Weighted Regression provides a suitable solution for smart-city traffic prediction because it focuses on traffic conditions that are most similar to the current situation. It can capture dynamic patterns caused by time, weather and changing road conditions while supporting real-time decision-making.